

Nota: Este material complementar, disponível em <https://prettore.github.io/lectures.html> representa uma cópia resumida de conteúdos bibliográficos disponíveis gratuitamente na Internet.

Vetores e Strings

Introdução.....	1
Vetores (Arrays).....	2
Definição e Características.....	2
Declaração e Inicialização.....	2
Acesso aos Elementos.....	2
Percorrendo Vetores.....	3
Aplicações de Vetores.....	3
Matrizes (Arrays Multidimensionais).....	3
Definição e Características.....	4
Declaração e Inicialização.....	4
Acesso aos Elementos.....	4
Percorrendo Matrizes.....	4
Aplicações de Matrizes.....	5
Strings.....	5
Definição e Características.....	5
Declaração e Inicialização.....	6
Acesso aos Caracteres.....	6
Operações com Strings.....	6
Representação de Strings em Memória.....	7
Codificação de Caracteres.....	7
Aplicações de Strings.....	8
Boas Práticas no Uso de Vetores e Strings.....	8
Conclusão.....	8
Referências.....	9

Introdução

Vetores e strings são estruturas de dados fundamentais em programação que permitem armazenar e manipular coleções de valores. Enquanto os vetores (também conhecidos como arrays) são utilizados para armazenar sequências de valores do mesmo tipo, as strings são especificamente destinadas ao armazenamento e manipulação de texto. Estas estruturas são essenciais para o desenvolvimento de algoritmos eficientes e para a solução de problemas que envolvem o processamento de múltiplos dados relacionados. Este material explora os conceitos, características e aplicações de vetores e strings, fornecendo uma base sólida para sua utilização em diferentes contextos de programação.

Vetores (Arrays)

Um vetor, também conhecido como array, é uma estrutura de dados indexada que permite armazenar uma coleção de valores do mesmo tipo sob um único nome de variável. Cada elemento do vetor é acessado através de um índice, que representa sua posição na estrutura. Os vetores são amplamente utilizados em programação para simplificar o código quando é necessário trabalhar com múltiplas variáveis do mesmo tipo.

Definição e Características

Um vetor pode ser definido como uma sequência de elementos do mesmo tipo, armazenados de forma contígua na memória. As principais características dos vetores incluem:

- Homogeneidade: Todos os elementos de um vetor devem ser do mesmo tipo (inteiros, reais, caracteres, etc.).
- Tamanho fixo: Em muitas linguagens, o tamanho do vetor é definido na sua declaração e não pode ser alterado durante a execução do programa.
- Acesso direto: Os elementos do vetor podem ser acessados diretamente através de seus índices, permitindo operações de leitura e escrita em tempo constante.
- Armazenamento contíguo: Os elementos são armazenados em posições adjacentes na memória, o que facilita o acesso sequencial.

Declaração e Inicialização

A forma de declarar e inicializar um vetor varia de acordo com a linguagem de programação, mas geralmente segue um padrão similar. Abaixo estão exemplos em algumas linguagens comuns:

Em C/C++	Em Java	Em Python
<pre>```c // Declaração int numeros[5]; // Declaração e inicialização int numeros[5] = {10, 20, 30, 40, 50}; // Inicialização sem especificar o tamanho int numeros[] = {10, 20, 30, 40, 50}; ```</pre>	<pre>```java // Declaração int[] numeros = new int[5]; // Declaração e inicialização int[] numeros = {10, 20, 30, 40, 50}; ```</pre>	<pre>```python # Declaração e inicialização numeros = [10, 20, 30, 40, 50] ```</pre>

Acesso aos Elementos

Os elementos de um vetor são acessados através de seus índices. Na maioria das linguagens de programação, a indexação começa em 0, ou seja, o primeiro elemento está na posição 0,

o segundo na posição 1, e assim por diante. Por exemplo:

```
```c
int numeros[5] = {10, 20, 30, 40, 50};
int primeiro = numeros[0]; // primeiro = 10
```

```
int terceiro = numeros[2]; // terceiro = 30
numeros[4] = 60; // Altera o quinto elemento para 60
...
```

É importante notar que tentar acessar um índice fora dos limites do vetor pode causar erros de execução ou comportamentos indefinidos, dependendo da linguagem.

## Percorrendo Vetores

Para processar todos os elementos de um vetor, geralmente utilizamos estruturas de repetição, como o loop for. Abaixo estão exemplos de como percorrer um vetor em diferentes linguagens:

Em C/C++	Em Java	Em Python
<pre>```c int numeros[5] = {10, 20, 30, 40, 50}; for (int i = 0; i &lt; 5; i++) {     printf("%d ", numeros[i]); } ```</pre>	<pre>```java int[] numeros = {10, 20, 30, 40, 50}; for (int i = 0; i &lt; numeros.length; i++) {      System.out.print(numeros[i] + " "); } // Ou usando o for-each for (int numero : numeros) {     System.out.print(numero + " "); } ```</pre>	<pre>```python numeros = [10, 20, 30, 40, 50] for numero in numeros:     print(numero, end=" ") ```</pre>

## Aplicações de Vetores

Os vetores são utilizados em uma ampla variedade de aplicações, incluindo:

- Armazenamento de coleções de dados: Como listas de nomes, notas de alunos, preços de produtos, etc.
- Implementação de estruturas de dados mais complexas: Como pilhas, filas e listas encadeadas.
- Processamento de dados em lote: Como cálculos estatísticos, ordenação e busca.
- Representação de séries temporais: Como históricos de preços, medições de temperatura, etc.
- Manipulação de imagens: Onde cada pixel pode ser representado como um elemento de um vetor bidimensional.

## Matrizes (Arrays Multidimensionais)

Uma matriz é uma extensão do conceito de vetor para múltiplas dimensões. Enquanto um vetor é uma estrutura unidimensional, uma matriz pode ter duas, três ou mais dimensões. A matriz mais comum é a bidimensional, que pode ser visualizada como uma tabela com linhas e colunas.

# Definição e Características

Uma matriz bidimensional pode ser definida como um vetor de vetores, onde cada elemento é acessado através de dois índices: um para a linha e outro para a coluna. As principais características das matrizes incluem:

- Homogeneidade: Assim como os vetores, todos os elementos de uma matriz devem ser do mesmo tipo.
- Dimensões fixas: Em muitas linguagens, as dimensões da matriz são definidas na sua declaração.
- Acesso direto: Os elementos da matriz podem ser acessados diretamente através de seus índices.
- Armazenamento contíguo: Os elementos são armazenados em posições adjacentes na memória, geralmente em ordem de linha ou coluna.

## Declaração e Inicialização

A forma de declarar e inicializar uma matriz varia de acordo com a linguagem de programação. Abaixo estão exemplos em algumas linguagens comuns:

Em C/C++	Em Java	Em Python
<pre>``c // Declaração int matriz[3][4]; // Declaração e inicialização int matriz[3][4] = {     {1, 2, 3, 4},     {5, 6, 7, 8},     {9, 10, 11, 12} }; ``</pre>	<pre>``java // Declaração int[][] matriz = new int[3][4]; // Declaração e inicialização int[][] matriz = {     {1, 2, 3, 4},     {5, 6, 7, 8},     {9, 10, 11, 12} }; ``</pre>	<pre>``python # Declaração e inicialização matriz = [     [1, 2, 3, 4],     [5, 6, 7, 8],     [9, 10, 11, 12] ] ``</pre>

### Acesso aos Elementos

Os elementos de uma matriz são acessados através de seus índices de linha e coluna. Por exemplo:

```
``c
int matriz[3][4] = {
 {1, 2, 3, 4},
```

```
 {5, 6, 7, 8},
 {9, 10, 11, 12}
};
int elemento = matriz[1][2]; // elemento = 7 (linha 1, coluna 2)
matriz[2][3] = 20; // Altera o elemento na linha 2, coluna 3 para 20
``
```

## Percorrendo Matrizes

Para processar todos os elementos de uma matriz, geralmente utilizamos loops aninhados. Abaixo estão exemplos de como percorrer uma matriz em diferentes linguagens:

Em C/C++	Em Java	Em Python
----------	---------	-----------

<pre> c int matriz[3][4] = {     {1, 2, 3, 4},     {5, 6, 7, 8},     {9, 10, 11, 12} }; for (int i = 0; i &lt; 3; i++) {     for (int j = 0; j &lt; 4; j++) {         printf("%d ", matriz[i][j]);     }     printf(" "); } </pre>	<pre> java int[][] matriz = {     {1, 2, 3, 4},     {5, 6, 7, 8},     {9, 10, 11, 12} }; for (int i = 0; i &lt; matriz.length; i++) {     for (int j = 0; j &lt; matriz[i].length; j++) {  System.out.print(matriz[i][j] + " ");     }     System.out.println(); } </pre>	<pre> python matriz = [     [1, 2, 3, 4],     [5, 6, 7, 8],     [9, 10, 11, 12] ] for linha in matriz:     for elemento in linha:         print(elemento, end=" ")     print() </pre>
------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

## Aplicações de Matrizes

As matrizes são utilizadas em uma ampla variedade de aplicações, incluindo:

- Representação de dados tabulares: Como tabelas de dados, planilhas, etc.
- Processamento de imagens: Onde cada pixel é representado como um elemento de uma matriz.
- Implementação de algoritmos matemáticos: Como multiplicação de matrizes, resolução de sistemas lineares, etc.
- Jogos: Para representar tabuleiros, mapas, etc.
- Simulações: Para representar grades de células em autômatos celulares, simulações de fluidos, etc.

## Strings

Strings são sequências de caracteres utilizadas para representar texto em programas de computador. Em muitas linguagens, as strings são implementadas como vetores de caracteres, mas com funcionalidades adicionais específicas para manipulação de texto.

## Definição e Características

Uma string pode ser definida como uma sequência de caracteres, geralmente armazenada como um vetor de caracteres. As principais características das strings incluem:

- Sequencialidade: Os caracteres são armazenados em sequência, permitindo acesso por índice.
- Tamanho variável: Em muitas linguagens modernas, as strings podem ter tamanho variável.
- Operações específicas: As linguagens geralmente fornecem operações específicas para strings, como concatenação, busca, substituição, etc.
- Representação de texto: As strings são utilizadas para representar texto em programas, incluindo mensagens, nomes, endereços, etc.

## Declaração e Inicialização

A forma de declarar e inicializar uma string varia de acordo com a linguagem de programação. Abaixo estão exemplos em algumas linguagens comuns:

Em C/C++	Em Java	Em Python
<pre>```c //      Declaração      e inicialização char nome[50] = "João"; // Ou char *nome = "João"; ```  ```cpp // Usando a classe string #include &lt;string&gt; std::string nome = "João"; ```</pre>	<pre>```java //      Declaração      e inicialização String nome = "João"; ```</pre>	<pre>```python #      Declaração      e inicialização nome = "João" ```</pre>

## Acesso aos Caracteres

Os caracteres de uma string podem ser acessados através de seus índices, assim como em um vetor. Por exemplo:

```
```c
char nome[50] =
"João";
char primeiraLetra
```

```
= nome[0]; // primeiraLetra = 'J'
```
```

Em linguagens como Java e Python, as strings são imutáveis, o que significa que não é possível alterar caracteres individuais diretamente. Nestes casos, é necessário criar uma nova string com as alterações desejadas.

## Operações com Strings

As linguagens de programação geralmente fornecem uma variedade de operações para manipulação de strings. Algumas das operações mais comuns incluem:

- Concatenação: Combinar duas ou mais strings em uma única string.

```
```python
nome = "João"
sobrenome = "Silva"
nomeCompleto = nome + " " + sobrenome # nomeCompleto = "João Silva"
```
```
- Comprimento: Obter o número de caracteres em uma string.

```
```python
nome = "João"
tamanho = len(nome) # tamanho = 4
```
```
- Busca: Encontrar a posição de uma substring dentro de uma string.

```
```python
texto = "Olá, mundo!"
```

```
posicao = texto.find("mundo") # posicao = 5
...

```

- Substituição: Substituir uma substring por outra.

```
```python
texto = "Olá, mundo!"
novoTexto = texto.replace("mundo", "amigo") # novoTexto = "Olá, amigo!"
...

```

- Extração de substrings: Obter uma parte de uma string.

```
```python
texto = "Olá, mundo!"
parte = texto[5:10] # parte = "mundo"
...

```

- Conversão de caso: Converter para maiúsculas ou minúsculas.

```
```python
texto = "Olá, mundo!"
maiusculas = texto.upper() # maiusculas = "OLÁ, MUNDO!"
minusculas = texto.lower() # minusculas = "olá, mundo!"
...

```

## Representação de Strings em Memória

A forma como as strings são representadas em memória varia de acordo com a linguagem de programação e a implementação específica. Em linguagens como C, as strings são representadas como vetores de caracteres terminados por um caractere nulo (""). Este caractere nulo serve como marcador de fim da string e não é considerado parte do conteúdo da string.

```
```c
char nome[5] = "João";
// Em memória: ['J', 'o', 'ã', 'o', '']
...

```

Em linguagens modernas como Java e Python, as strings são objetos mais complexos que encapsulam o vetor de caracteres e fornecem métodos para manipulação. Nestas linguagens, as strings geralmente são imutáveis, o que significa que qualquer operação que modifique a string na verdade cria uma nova string.

Codificação de Caracteres

As strings são armazenadas em memória como sequências de bytes, e a forma como os caracteres são mapeados para bytes é determinada pela codificação de caracteres utilizada. Algumas das codificações mais comuns incluem:

- ASCII: Uma codificação de 7 bits que representa 128 caracteres, incluindo letras do alfabeto inglês, dígitos e alguns símbolos.
- ISO-8859-1 (Latin-1): Uma extensão de 8 bits do ASCII que inclui caracteres adicionais utilizados em línguas europeias ocidentais.

- UTF-8: Uma codificação de tamanho variável que pode representar qualquer caractere Unicode. Os caracteres ASCII são representados por 1 byte, enquanto outros caracteres podem ocupar de 2 a 4 bytes.
- UTF-16: Uma codificação que utiliza 2 ou 4 bytes por caractere.

A escolha da codificação afeta como os caracteres especiais, como letras acentuadas e símbolos, são representados e manipulados em strings.

Aplicações de Strings

As strings são utilizadas em uma ampla variedade de aplicações, incluindo:

- Processamento de texto: Como editores de texto, processadores de palavras, etc.
- Comunicação: Como mensagens, e-mails, chats, etc.
- Armazenamento de dados: Como nomes, endereços, descrições, etc.
- Processamento de linguagem natural: Como análise de sentimentos, tradução automática, etc.
- Web: Como HTML, CSS, JavaScript, URLs, etc.

Boas Práticas no Uso de Vetores e Strings

Para utilizar vetores e strings de forma eficiente e segura, é recomendável seguir algumas boas práticas:

- Verificação de limites: Sempre verifique se os índices utilizados estão dentro dos limites do vetor ou string para evitar acessos inválidos.
- Inicialização: Inicialize vetores e strings antes de utilizá-los para evitar valores indeterminados.
- Alocação de memória: Em linguagens que exigem alocação manual de memória, certifique-se de alocar espaço suficiente para os dados e liberar a memória quando não for mais necessária.
- Uso de funções da biblioteca padrão: Utilize as funções e métodos fornecidos pela biblioteca padrão da linguagem para manipulação de vetores e strings, pois elas geralmente são otimizadas e testadas.
- Tratamento de erros: Implemente tratamento de erros adequado para lidar com situações como índices inválidos, memória insuficiente, etc.
- Documentação: Documente o propósito e o uso de vetores e strings em seu código para facilitar a manutenção.

Conclusão

Vetores e strings são estruturas de dados fundamentais em programação que permitem armazenar e manipular coleções de valores e texto, respectivamente. Os vetores são utilizados para armazenar sequências de valores do mesmo tipo, enquanto as strings são especificamente destinadas ao armazenamento e manipulação de texto. Ambas as estruturas são essenciais para o desenvolvimento de algoritmos eficientes e para a solução de problemas que envolvem o processamento de múltiplos dados relacionados.

Neste material, exploramos os conceitos, características e aplicações de vetores e strings, fornecendo uma base sólida para sua utilização em diferentes contextos de programação. Compreender essas estruturas é fundamental para qualquer programador, pois elas são utilizadas em praticamente todos os tipos de aplicações, desde simples scripts até sistemas complexos.

Ao dominar o uso de vetores e strings, você estará equipado com ferramentas poderosas para resolver uma ampla variedade de problemas de programação de forma eficiente e elegante.

Referências

-  Livros e Apostilas
 - CORMEN, T. H. Introduction to Algorithms. MIT Press.
 - GOODRICH, M. Data Structures and Algorithms in Python.
 - Tenenbaum, A. M. Estruturas de Dados e Algoritmos em C
 - P. Feofiloff. Algoritmos em Linguagem C. Campus-Elsevier, 1a. edição, 2009 H. M. Deitel, P. J. Deitel. C - Como Programar, 6a. edição, Pearson Education, 2011.
 - B. W. Kernighan, D. M. Ritchie. The C Programming Language, 2a. edição, Prentice-Hall, 1988 [Tradução: C - A Linguagem de Programação. Editora Campus, 1989].
 - J. L. Szwarcfiter, L. Markenzon. Estruturas de Dados e seus Algoritmos, 3a. edição, Editora LTC, 2010.
 - W. Celes, R. Cerqueira, J.L. Rangel. Introdução a Estruturas de Dados, 1a. edição, Editora Campus, 2004.
 - N. Ziviani. Projeto de Algoritmos com Implementações em Pascal e C, 3a. edição, Editora Cengage Learning, 2011.
 - T. Cormen, C. Leiserson, R. Rivest, C. Stein. Algoritmos - Teoria e Prática, 3a. edição, Editora Campus, 2012.
 - R. Sedgewick, K. Wayne. Algorithms, 4a. edição, Addison -Wesley, 2011.
 - A. Kelley, I. Pohl. A Book on C, 4a. edição, Addison Wesley, 1998.
-  Recursos Online
 - Programação para Iniciantes: Vetores, Matrizes, Arrays, Strings e afins - <https://blog.matheus.io/pt/desenvolvimento/programacao-o-que-sao-vetores-matrizes-arrays-strings/>
 - O que são Vetores e Matrizes (arrays) - <https://dicasdeprogramacao.com.br/o-que-sao-vetores-e-matrizes-arrays/>
 - Vetores – arrays em linguagem C - <https://linguagemc.com.br/vetores-ou-arrays-em-linguagem-c/>
 - Strings e cadeias de caracteres - <https://www.ime.usp.br/~pf/algoritmos/aulas/string.html>
 - <Direto ao Ponto 47> Estruturas de Dados - Cadeias de caracteres (Strings) - <https://www.dio.me/articles/direto-ao-ponto-47-cadeias-de-caracteres-strings>
 - W3Schools - Arrays
 - GeeksforGeeks - Arrays and Strings
-  Vídeos e Cursos
 - Curso em Vídeo - Vetores e Matrizes
 - Programação de Computadores - Vetores e Strings - UFOP
 - Khan Academy - Introdução à Programação

Isenção de Responsabilidade:

Os autores deste documento não reivindicam a autoria do conteúdo original compilado das fontes mencionadas. Este documento foi elaborado para fins educativos e de referência, e todos os créditos foram devidamente atribuídos aos respectivos autores e fontes originais.

Qualquer utilização comercial ou distribuição do conteúdo aqui compilado deve ser feita com a devida autorização dos detentores dos direitos autorais originais. Os compiladores deste documento não assumem qualquer responsabilidade por eventuais violações de direitos autorais ou por quaisquer danos decorrentes do uso indevido das informações contidas neste documento.

Ao utilizar este documento, o usuário concorda em respeitar os direitos autorais dos autores originais e isenta os compiladores de qualquer responsabilidade relacionada ao conteúdo aqui apresentado.